Data Structures

Lecture 14: Linked List

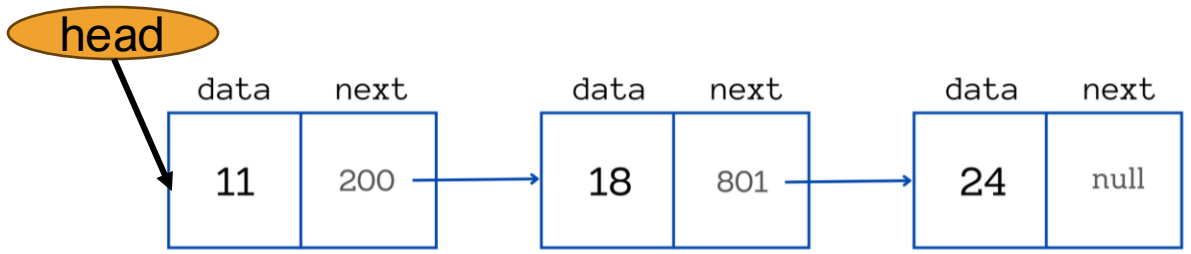
Dr. Shabnam Samima

Content

- Searching

Question 1

Consider that the linked list as depicted in the figure is given:



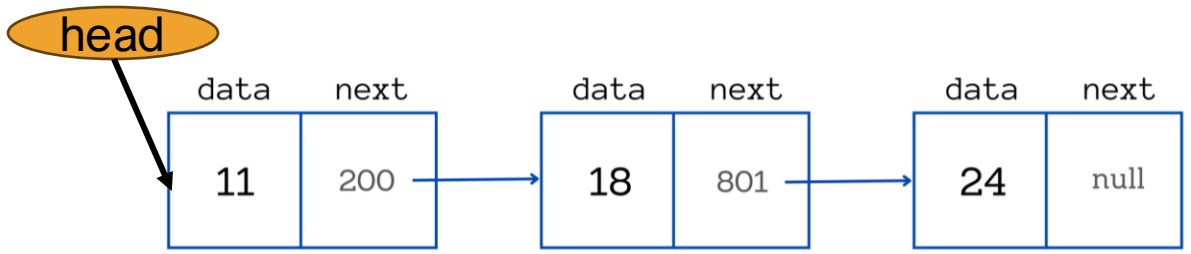
- Write a C Function or Pseudocode to print data in the linked list

```
// Define a node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to print the linked list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Question 1

Consider that the linked list as depicted in the figure is given:

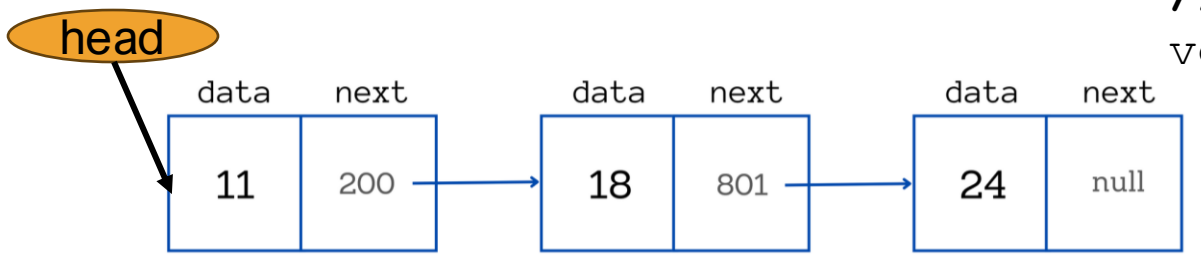


- Write a C Function or Pseudocode to print data in the linked list

```
// Function to print the linked list
void printList(struct Node* head) // The function name is printList. It takes one argument: head, which is a pointer to the first node of the linked list; struct Node* means it points to a structure of type Node
   {
      struct Node* temp = head; // A temporary pointer temp is created. It is initialized with head. We use temp so that we don't modify the original head pointer while traversing.
      while (temp != NULL) {
          printf("%d -> ", temp->data);
          temp = temp->next;
      }
      printf("NULL\n");
}
```

Question 2

Consider that the linked list as depicted in the figure is given:



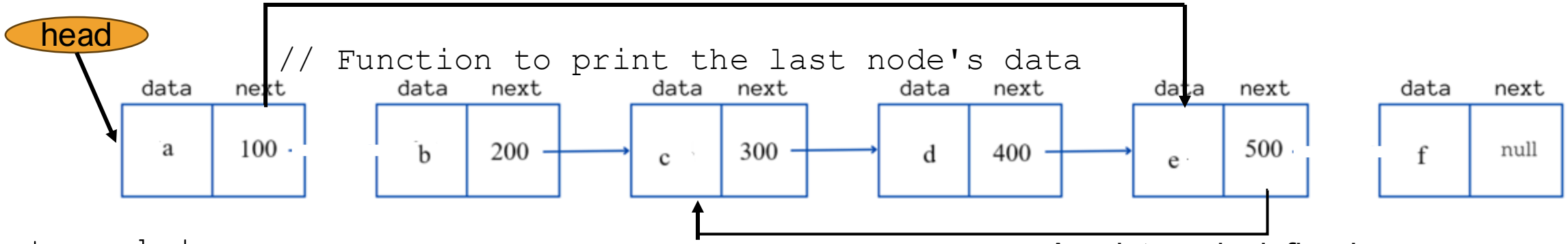
- Write a C Function or Pseudocode to print data of the last node

```
// Define a node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to print the last node
void printLastNode(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;    }
        struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;      }
    printf("Last node data: %d\n", temp->data);
}
```

Question 3

Consider that the linked list as depicted in the figure is given. What is the output of the following code

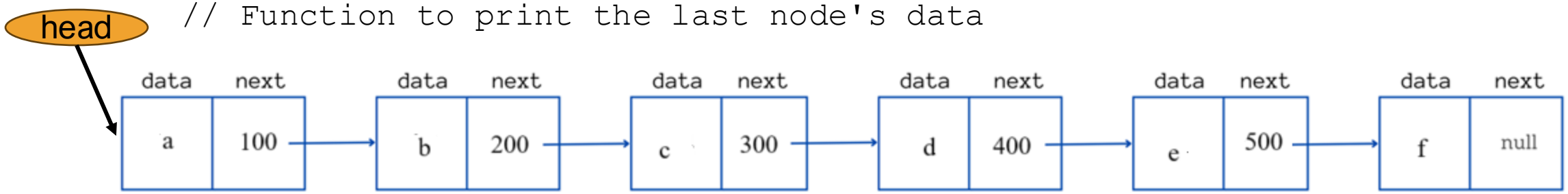



```
Struct node* p
p=head→next→next
head→ next= p→next→next
p→next→next→next=p
printf(“%c”,head →next→next→next→ data)
```

A pointer p is defined
p= 200
Fifth node becomes Second node
Third node coming after fifth node
d

Question 4

Consider that the linked list as depicted in the figure is given. What is the output of the following code




```
// Function to print the last node's data
Struct node* p
p=head→next→next
head→ next= p→next→next
p→next→next→next →next=p
printf(“%c”,head →next→next→next→ data)
```

A pointer p is defined
p= 200
Fifth node becomes Second node
Third node coming after Sixth node
c

Linked List: Searching

The idea is to traverse all the nodes of the linked list, starting from the **START**. While traversing, if we find a node whose value is equal to key then print the value, otherwise print "No" to indicate data is not present.

- One of the two case may come for searching
  - List is sorted
  - List is unsorted

Searching: Unsorted List

Consider that the data in LIST is unsorted.

- Traverse through the list using a pointer variable PTR
- Compare ITEM to be searched with INFO[PTR].
- If found terminate search
- Otherwise go to the next node by updating pointer PTR to PTR := LINK[PTR]

Before we update the pointer PTR by PTR := LINK[PTR] we require two tests.

- First, we have to check to see whether we have reached the end of the list; i.e., whether PTR = NULL
- If not, then we check to see whether INFO[PTR] = ITEM

Searching: Unsorted List

Algorithm

```
SEARCH(INFO, LINK, START, ITEM, LOC)
1. Set PTR := START.
2. Repeat next step while PTR ≠ NULL:
      If ITEM = INFO[PTR], then:
            Set LOC := PTR, and Exit.
      Else:
            Set PTR := LINK[PTR]. [PTR now points to the next node.]
3. [Search is unsuccessful.] Set LOC := NULL
4. Exit
```

Time Complexity O(n)

Searching: Sorted List

Consider that the data in LIST is sorted.

- Traverse through the list using a pointer variable PTR
- Compare ITEM to be searched with INFO[PTR].
- We can terminate search if
  - INFO[PTR] exceeds ITEM to be searched
  - we have reached the end of the list; i.e., whether PTR = NULL

Searching: Sorted List

Algorithm

```
SRCHSL(INFO, LINK, START, ITEM, LOC)
1. Set PTR := START.
2. Repeat next step while PTR ≠ NULL:
      If ITEM > INFO[PTR], then:
            Set PTR := LINK[PTR]. [PTR now points to next node.]
      Else if ITEM = INFO[PTR], then:
            Set LOC := PTR, and Exit. [Search is successful.]
      Else:
            Set LOC := NULL, and Exit. [ITEM now exceeds INFO[PTR].]
1. Set LOC := NULL.
2. Exit.
```

Time Complexity O(n)